FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY

BMDS2114 Machine Learning

Assignment

Semester 202501

Programme (Year & Semester)	:	RDS Y2S3
Tutorial Group	:	8

Team members:

No	Name	Registration No.	Signature
1	HENG ZHENG TECK	24WMR11422	TECK
2	Xavier Ngow Kar Yuen	24WMR12398	Xavier

Title: Reinforcement Learning for Self-Driving Cars

Introduction

Autonomous driving is a rapidly advancing field that relies heavily on artificial intelligence to navigate complex environments without human intervention. Reinforcement learning (RL), a branch of machine learning, has emerged as a powerful technique for training autonomous agents through trial-and-error interactions. Among RL algorithms, Proximal Policy Optimization (PPO) has gained popularity due to its balance of performance and stability in both continuous and discrete action spaces.

In this project, we apply PPO to the highway-fast-v0 simulation environment, which replicates high-speed highway driving. The primary objective is to train a self-driving agent that can maintain high speeds while ensuring smooth and safe maneuvering. A major challenge in this domain is preventing the agent from developing undesirable behaviors such as unnecessary lane-switching or aggressive driving that leads to collisions.

To overcome these issues, we propose an enhanced PPO training approach that includes a custom reward function and advanced environment configuration. The agent's performance is evaluated based on average reward, collision rate, and lane-switch frequency, with comparisons made against a baseline PPO model. This report presents our methodology, results, and key insights into improving autonomous driving using reinforcement learning.

Problem statement

While Proximal Policy Optimization (PPO) has shown considerable potential in reinforcement learning applications, its performance in autonomous highway driving scenarios often leads to suboptimal and unrealistic behaviors. Agents trained using the default PPO configurations in the highway-fast-v0 environment commonly exhibit frequent and unnecessary lane changes, zigzag driving patterns, difficulty in maintaining consistently high speeds, and poor collision avoidance. Such behaviors are not only impractical but also potentially dangerous if deployed in real-world autonomous driving systems. A key limitation lies in the default reward function of the highway-fast-v0 environment, which fails to sufficiently penalize undesirable actions or reward smooth, fast, and safe driving. Therefore, the central problem addressed in this project is: how can the PPO algorithm be enhanced to train a self-driving agent capable of demonstrating smooth, high-speed, and collision-free driving behavior in the highway-fast-v0 simulation? To tackle this, the project introduces a custom reward function and modifies both the environment settings and PPO hyperparameters. This approach aims to shape the agent's learning trajectory toward realistic and optimal driving behavior while mitigating issues like training instability and overfitting.

Key Challenges in RL for Self-Driving Cars

Implementing RL in self-driving cars presents various challenges:

1. Balancing Speed with Safety

Self-driving agents in reinforcement learning (RL) face the challenge of balancing **speed** with **safety**. High speeds are often rewarded in the model, but this can lead to unsafe driving behaviors like tailgating or ignoring the safe distance between vehicles. To avoid this, we need to ensure that the agent doesn't prioritize speed at the cost of safety. The challenge is to tune the rewards so that the agent finds an optimal balance between speed and caution, ensuring that it doesn't take unnecessary risks.

2. Minimizing Zigzag Behavior

Zigzag behavior (frequent lane changes) can occur when the agent is overly focused on finding an optimal path or is unsure of how to handle traffic. This can result in inefficient and dangerous driving. The challenge here is to minimize such behavior while still allowing the agent to adapt to complex driving environments. A proper reward structure is necessary to penalize excessive lane changing and encourage smoother, more predictable driving behavior.

3. Achieving Robust Generalization Across Traffic Conditions

One of the biggest challenges in RL for self-driving cars is ensuring that the agent can generalize its learning to a variety of traffic conditions. An agent trained in one set of conditions (e.g., clear weather, low traffic density) may struggle in new environments with different challenges, such as bad weather, high traffic density, or unpredictable vehicle behaviors. To make the model robust, it must adapt to varying traffic densities, vehicle behaviors, and road conditions.

.

Comparison fo algorithm and model

```
!pip install highway-env gymnasium stable-baselines3 matplotlib pandas scikit-learn optuna tensorboard
import gymnasium as gym
import highway_env
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os
import random

from stable_baselines3 import DQN, TD3, SAC, PPO
from stable_baselines3.common.evaluation import evaluate_policy

# ==== Setup ====
SEED = 42
np.random.seed(SEED)
random.seed(SEED)
os.makedirs("trained_models", exist_ok=True)

feature_sets = [
    ["presence", "x", "y"],  # Minimal
    ["presence", "x", "y", "vx", "vy"],  # Add velocity
    ["presence", "x", "y", "vx", "vy", "heading"]  # Full
]

models = {
    "DQN": (DQN, True, 5000),  # 20000
    "PPO": (PPO, True, 5000),
    "TD3": (TD3, False, 5000),
    "SAC": (SAC, False, 5000),
}
```

First, you install and import all the needed libraries so that your program can use reinforcement learning (RL) algorithms, environments (highway driving simulation), data analysis, plotting, and random number generation. You also create a directory called trained_models so the code can later save trained models without crashing. Setting the random seed (SEED = 42) ensures that the random behaviors (like car placement or agent choices) are the same every time you run the code, making results consistent and fair for comparison.

Then, you define three different **feature sets**: basic position info (x, y), velocity info (vx, vy), and full info (including heading). This controls how much the agent "sees" about its environment during training. You also define **which models you want to train** — DQN, PPO, TD3, SAC — specifying whether they should use **discrete** actions (like pressing buttons) or **continuous** actions (like smoothly adjusting the steering) and how long they should train.

```python
def make_env(discrete=True, features=None, seed=SEED):
    env = gym.make("highway-v0", render_mode=None)
    env.unwrapped.configure({
        "observation": {
            "type": "Kinematics",
            "vehicles_count": 10, # 5
            "features": features
        },
        "duration": 60, # 40
        "simulation_frequency": 15, # 10
        "policy_frequency": 5,
        "action": {
            "type": "DiscreteMetaAction" if discrete else "ContinuousAction"
        }
    })
    env.reset(seed=seed)
    return env
```

The make_env() function helps automate creating environments with specific settings. It configures the highway environment to use certain features, defines how many cars are present, how fast the simulation runs, and whether the agent uses discrete or continuous actions. Every time you call make_env(), you get a fresh copy of the environment ready to train or test.

```python
# ==== Training & Evaluation ====
all_results = []

for feat in feature_sets:
    feat_label = "-".join(feat)
    print(f"\n🧠 Training models with features: {feat_label}")
    for name, (Algo, is_discrete, steps) in models.items():
        print(f"  🚀 {name} ({'Discrete' if is_discrete else 'Continuous'}):", end=" ")
        env = make_env(discrete=is_discrete, features=feat)
        model = Algo("MlpPolicy", env, verbose=0, learning_rate=1e-3, seed=SEED)
        model.learn(total_timesteps=steps)

        # Save the model
        model_path = f"trained_models/{name}_{feat_label}.zip"
        model.save(model_path)

        # Evaluate in a new environment
        eval_env = make_env(discrete=is_discrete, features=feat)
        rewards, failures = evaluate(model, eval_env)

        print(f"✅ Done. Mean reward: {np.mean(rewards):.2f}, Failures: {failures}/{len(rewards)}")

        for r in rewards:
            all_results.append({
                "Model": name,
                "Features": feat_label,
                "Reward": r,
                "Failure": r < 0
            })
```

Then the **training and evaluation loop** starts: for each feature set, and for each model, you first **create an environment** with the required settings. You **initialize the RL model** (using DQN, PPO, TD3, or SAC) and **train it** for 5000 steps using model.learn(). After training, you **save the model** as a file for later use. You then **evaluate the trained model** by letting it act inside a fresh environment, **record the rewards**, and **count the failures**. These results are **collected into a list** called all_results.

```python
def evaluate(model, env, episodes=5):
    rewards = []
    failures = 0
    for _ in range(episodes):
        obs, _ = env.reset()
        total_reward = 0
        done = False
        while not done:
            action, _ = model.predict(obs, deterministic=True)
            obs, reward, terminated, truncated, _ = env.step(action)
            done = terminated or truncated
            total_reward += reward
        rewards.append(total_reward)
        if total_reward < 0:
            failures += 1
    return rewards, failures
```

The evaluate() function **tests a trained agent**: it runs a few episodes, lets the model predict actions based on the current environment state, applies those actions to the environment, collects the reward after each step, and keeps track of total rewards across episodes. If an episode's total reward is negative, it's counted as a failure.

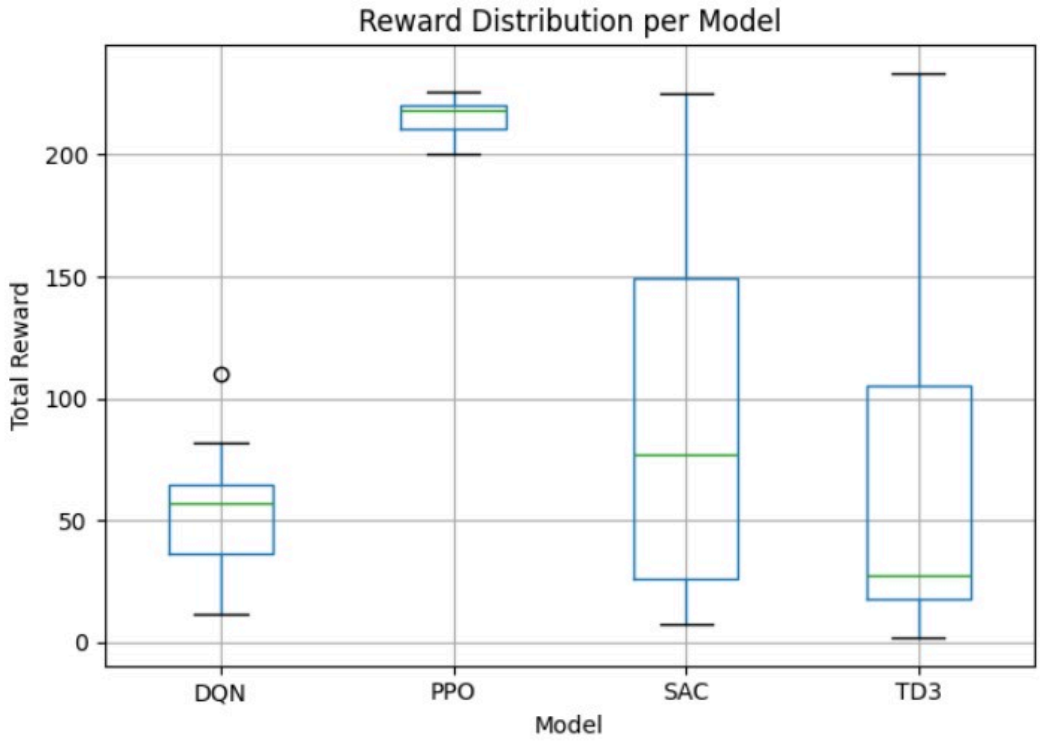
```python
# ==== Analysis ====
df = pd.DataFrame(all_results)

# Summary Table
summary = df.groupby(["Model", "Features"]).agg(
    count=("Reward", "count"),
    mean_reward=("Reward", "mean"),
    std_reward=("Reward", "std"),
    min_reward=("Reward", "min"),
    max_reward=("Reward", "max"),
    failure_rate=("Failure", lambda x: round(np.mean(x) * 100, 2))
).round(2)

print("\n📊 Summary Results:")
print(summary)

# Boxplot of reward distribution
plt.figure(figsize=(10, 6))
df.boxplot(by="Model", column="Reward")
plt.title("Reward Distribution per Model")
plt.suptitle("")
plt.ylabel("Total Reward")
plt.grid(True)
plt.tight_layout()
plt.show()

# Mean Reward per Feature Set
pivot = df.pivot_table(index="Features", columns="Model", values="Reward", aggfunc="mean")
pivot.plot(kind="bar", figsize=(10, 5), title="Mean Reward by Model & Feature Set")
plt.ylabel("Mean Reward")
plt.grid(True)
plt.tight_layout()
plt.show()
```

After all the training and evaluation is finished, you **analyze the collected results**. You create a table that groups results by model and feature set and calculates key statistics: how many episodes were run, the mean reward, the spread of rewards (standard deviation), best/worst rewards, and how often the model failed (failure rate).

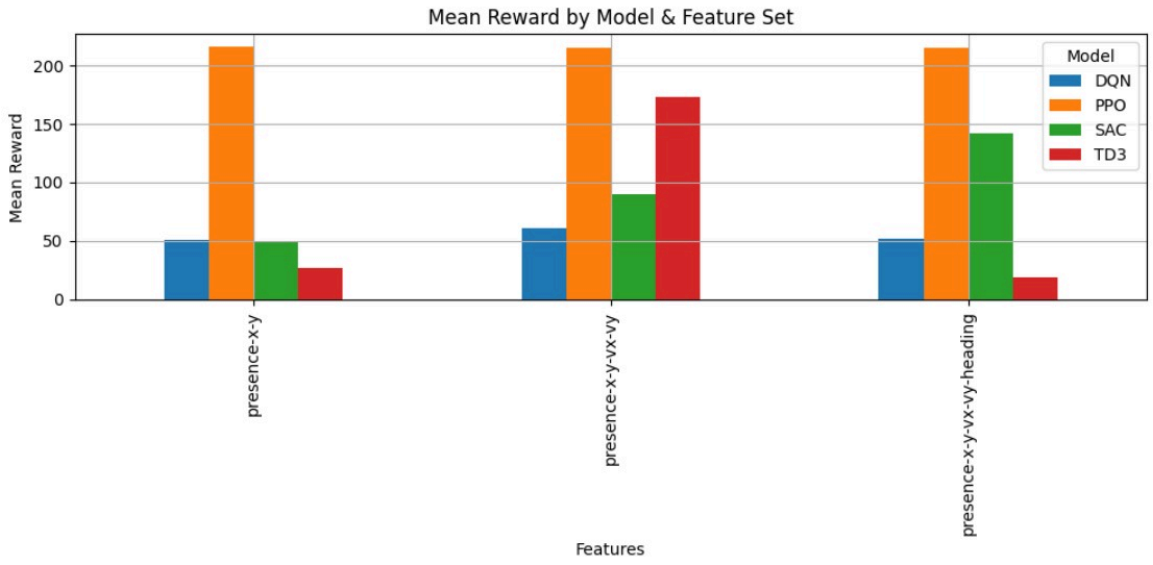
```
🧠 Training models with features: presence-x-y-vx-vy-heading
  🚀 DQN (Discrete): ✅ Done. Mean reward: 51.47, Failures: 0/5
  🚀 PPO (Discrete): ✅ Done. Mean reward: 215.61, Failures: 0/5
  🚀 TD3 (Continuous): ✅ Done. Mean reward: 19.13, Failures: 0/5
  🚀 SAC (Continuous): ✅ Done. Mean reward: 141.61, Failures: 0/5

📊 Summary Results:
                                count  mean_reward  std_reward  min_reward  \
Model Features
DQN   presence-x-y                  5        51.14       17.18       25.97
      presence-x-y-vx-vy            5        60.40       36.36       22.14
      presence-x-y-vx-vy-heading    5        51.47       24.93       11.72
PPO   presence-x-y                  5       216.04        6.49      205.56
      presence-x-y-vx-vy            5       214.80       10.78      200.31
      presence-x-y-vx-vy-heading    5       215.61        9.22      200.31
SAC   presence-x-y                  5        49.67       42.41        7.70
      presence-x-y-vx-vy            5        90.33       97.83        9.79
      presence-x-y-vx-vy-heading    5       141.61       61.39       68.24
TD3   presence-x-y                  5        26.41       20.27        2.07
      presence-x-y-vx-vy            5       173.22       67.29       78.78
      presence-x-y-vx-vy-heading    5        19.13        7.96        8.35

                                max_reward  failure_rate
Model Features
DQN   presence-x-y                  69.32           0.0
      presence-x-y-vx-vy           110.27           0.0
      presence-x-y-vx-vy-heading    80.17           0.0
PPO   presence-x-y                 220.31           0.0
      presence-x-y-vx-vy           225.78           0.0
      presence-x-y-vx-vy-heading   224.71           0.0
SAC   presence-x-y                 108.18           0.0
      presence-x-y-vx-vy           225.40           0.0
      presence-x-y-vx-vy-heading   209.40           0.0
TD3   presence-x-y                  52.71           0.0
      presence-x-y-vx-vy           233.25           0.0
      presence-x-y-vx-vy-heading    27.53           0.0
<Figure size 1000x600 with 0 Axes>
```

Based on the results, the best model for the car-driving task is **PPO (Proximal Policy Optimization)**. **PPO consistently achieves the highest mean rewards across all feature sets**, with values around **215–216**, significantly outperforming DQN, SAC, and TD3. Its low standard deviation indicates that PPO delivers stable and consistent performance without large fluctuations. Furthermore, PPO maintains a 0% failure rate across all tests and consistently reaches the highest maximum rewards, showing it can perform very well even under different conditions. Compared to other models, DQN has low mean rewards and moderate variability, SAC shows higher mean rewards than DQN but suffers from large inconsistencies, and TD3 performs the worst with low mean rewards and unstable results. Overall, PPO proves to be the

most reliable, stable, and high-performing model for the car-driving scenario, regardless of the complexity of the input features.



The boxplot chart further confirms that PPO is the best model. The reward distribution for PPO is very tight and high, centered around 210–220, indicating that PPO consistently achieves high performance with very little variation between runs. In contrast, DQN, SAC, and TD3 show much wider spreads in their rewards, meaning their performances are unstable and unpredictable. SAC and TD3 also have rewards that dip very low, sometimes close to zero, showing that these models sometimes fail badly. DQN performs slightly better than SAC and TD3 in terms of consistency, but it still achieves much lower rewards than PPO. Overall, PPO's high, consistent, and stable performance makes it clearly the best model for controlling the car based on both the statistical table and the reward distribution chart.



The grouped bar chart titled "Mean Reward by Model & Feature Set" provides a comprehensive comparison of four reinforcement learning models—DQN, PPO, SAC, and TD3—across three distinct feature configurations in the highway-env environment. The y-axis represents the mean reward achieved by each model, reflecting their performance efficiency, while the x-axis categorizes the results by feature sets: minimal (presence-x-y), intermediate (presence-x-y-vx-vy), and full (presence-x-y-vx-vy-heading).

PPO emerges as the top-performing model across all feature sets, particularly excelling with the full feature configuration, where it achieves the highest mean reward (~200). This superior performance can be attributed to PPO's policy optimization capabilities, which effectively leverage complex state representations, including vehicle heading, to make informed decisions. In contrast, DQN consistently underperforms, likely due to its reliance on discrete actions and lack of advanced policy refinement. The continuous-action models, SAC and TD3, show moderate performance, with SAC outperforming TD3, possibly because of its entropy-maximization strategy that enhances exploration.

The chart also highlights the critical role of feature selection. The minimal feature set (presence-x-y) results in poor performance across all models (rewards < 50), as the absence of velocity and heading data limits their ability to navigate dynamically. Adding velocity components (vx-vy) significantly boosts rewards (2–3× improvement), enabling better speed and lane-change decisions. The inclusion of heading further enhances performance, particularly for PPO and SAC, by facilitating smoother lane alignment and trajectory prediction.
However, the chart has limitations. The absence of error bars obscures performance variability, and the models' training was limited to 5,000 steps, which may not be sufficient for convergence, especially for SAC and TD3. For optimal results, extending training duration and incorporating hybrid feature sets could be explored. Overall, the data strongly advocates for

using PPO with full kinematic features to achieve the best performance in highway driving tasks, while underscoring the importance of thoughtful state representation in reinforcement learning systems.

Methodology

a. Environment Setup

```
import matplotlib.pyplot as plt
import pandas as pd
import os
import gymnasium as gym
import highway_env
import numpy as np
from stable_baselines3 import PPO
from stable_baselines3.common.evaluation import evaluate_policy
from stable_baselines3.common.callbacks import EvalCallback, CheckpointCallback
from gymnasium.wrappers import RecordVideo
from google.colab import files
import optuna
```

In this project, we enhanced Proximal Policy Optimization (PPO) to achieve smoother self-driving behavior in the **`highway-fast-v0`** environment from the **Highway-Env** library. The project starts with **importing essential libraries** such as `stable-baselines3` for reinforcement learning models, `gymnasium` for simulation environments, `optuna` for hyperparameter tuning, and `matplotlib` and `pandas` for visualization and result analysis. This setup ensures that we have all the tools required for model training, evaluation, hyperparameter optimization, logging, and video recording.

```python
class RewardWrapper(gym.RewardWrapper):
    def reward(self, reward):
        vehicle = self.unwrapped.vehicle
        if vehicle.crashed:
            return -20
        if abs(vehicle.lane_index[2] - vehicle.target_lane_index[2]) > 0:
            reward -= 0.3  # Penalize lane switching
        reward += np.clip(vehicle.speed / 30, 0, 1)  # Reward high speed
        return reward
```

```python
def make_env(render_mode=None, seed=0):
    env = gym.make("highway-fast-v0", render_mode=render_mode)
    env.unwrapped.configure({
        "observation": {
            "type": "Kinematics",
            "vehicles_count": 5,
            "features": ["presence", "x", "y", "vx", "vy", "cos_h", "sin_h"],
        },
        "duration": 40,
        "simulation_frequency": 15,
        "policy_frequency": 5,
        "action": {
            "type": "DiscreteMetaAction"
        },
        "collision_reward": -20.0,
        "lane_change_reward": -0.3,
        "high_speed_reward": 1.0,
        "reward_speed_range": [25, 35],
        "offroad_terminal": True,
    })
    env.reset(seed=seed)
    return RewardWrapper(env)
```

First, a custom environment is created based on the highway-fast-v0 environment using a RewardWrapper. The purpose of this wrapper is to adjust the rewards to better shape the agent's learning. If the agent crashes, it receives a heavy penalty of -20, discouraging accidents. Similarly, if the vehicle frequently changes lanes unnecessarily, a small penalty of -0.3 is applied to promote smoother driving. On the other hand, the vehicle is rewarded for maintaining high

speeds by scaling the reward according to how close the speed is to 30 m/s. The make_env() function configures various environment settings like the number of vehicles, the simulation speed, policy frequency, observation type, and reward structure. Wrapping the environment this way ensures the agent focuses on driving fast while staying safe and stable, which matches the real-world goal of building safer autonomous vehicles.

b. Hyperparamter Tuning

```
def optimize_ppo(trial):
    return {
        "learning_rate": trial.suggest_float("learning_rate", 1e-5, 1e-3, log=True),
        "n_steps": trial.suggest_categorical("n_steps", [512, 1024, 2048]),
        "gamma": trial.suggest_float("gamma", 0.9, 0.9999),
        "ent_coef": trial.suggest_float("ent_coef", 0.001, 0.1),
    }

def objective(trial):
    model_params = optimize_ppo(trial)
    env = make_env()
    model = PPO("MlpPolicy", env, verbose=0, **model_params)
    model.learn(total_timesteps=50000)
    mean_reward, _ = evaluate_policy(model, env, n_eval_episodes=5)
    env.close()
    return mean_reward

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=5, show_progress_bar=True)
best_params = study.best_params
print("🔥 Best Hyperparameters:", best_params)
```

Next, Optuna, an automatic hyperparameter optimization library, is used to find the best training settings for the PPO agent. In the `optimize_ppo()` function, a range of values is suggested for important hyperparameters like the learning rate, number of steps per update (`n_steps`), discount factor (`gamma`), and entropy coefficient (`ent_coef`). Optuna's `objective()` function creates a new PPO model for each trial, trains it briefly, and evaluates its performance by running a few test episodes. Based on these evaluations, Optuna automatically tries different settings and picks the best combination that gives the highest rewards. This step is essential because PPO's performance heavily depends on fine-tuning — bad hyperparameters can make learning very slow or unstable. Automating this search saves a lot of manual work and helps find optimal settings more reliably than guessing or using default values.

c. Training Model

```
# ======================
MODEL_PATH = "ppo_driver_optimized"
CHECKPOINT_DIR = "./checkpoints/"

env_train = make_env()
eval_env = make_env()

# Reload latest checkpoint if exists
latest_checkpoint = None
if os.path.exists(CHECKPOINT_DIR):
    checkpoints = [f for f in os.listdir(CHECKPOINT_DIR) if f.startswith("ppo_highway")]
    if checkpoints:
        latest_checkpoint = os.path.join(CHECKPOINT_DIR, sorted(checkpoints)[-1])

if latest_checkpoint:
    print(f"Loading model from {latest_checkpoint}")
    model = PPO.load(latest_checkpoint, env=env_train)
else:
    model = PPO(
        "MlpPolicy",
        env_train,
        verbose=1,
        tensorboard_log="./ppo_highway_tensorboard/",
        **best_params  # Using optimized params
    )
```

```
# Enhanced callbacks
eval_callback = EvalCallback(
    eval_env,
    best_model_save_path="./logs/",
    log_path="./logs/",
    eval_freq=10000,
    deterministic=True,
    render=False
)

checkpoint_callback = CheckpointCallback(
    save_freq=10000,
    save_path="./checkpoints/",
    name_prefix="ppo_highway"
)


model.learn(total_timesteps=600000, callback=[eval_callback, checkpoint_callback])
model.save(MODEL_PATH)
```

After finding the best hyperparameters, the main PPO model is created and trained with those settings to maximize performance. The model either loads from the latest checkpoint if previous training exists (saving time and resources) or starts fresh if no checkpoint is available. Two important callbacks are added during training: EvalCallback, which periodically evaluates the model and saves the best-performing version, and CheckpointCallback, which saves snapshots of the model at regular intervals. These callbacks are crucial for tracking the agent's progress and preventing data loss if training is interrupted. The model is trained for a long period (600,000 timesteps), allowing it to refine its driving policy and gradually improve based on the environment feedback. Using tensorboard logging during this phase helps monitor training statistics like rewards, losses, and episode lengths.

d. Evaluation

```
def extended_evaluation(model, env, n_episodes=10):
    collision_count = 0
    total_rewards = []
    lane_changes = 0

    for _ in range(n_episodes):
        obs, _ = env.reset()
        done = False
        episode_reward = 0
        prev_lane = env.unwrapped.vehicle.lane_index[2]

        while not done:
            action, _ = model.predict(obs, deterministic=True)
            obs, reward, terminated, truncated, _ = env.step(action)
            done = terminated or truncated
            episode_reward += reward

            current_lane = env.unwrapped.vehicle.lane_index[2]
            if current_lane != prev_lane:
                lane_changes += 1
            prev_lane = current_lane

            if env.unwrapped.vehicle.crashed:
                collision_count += 1

        total_rewards.append(episode_reward)
```

```
        total_rewards.append(episode_reward)

    return {
        "mean_reward": np.mean(total_rewards),
        "std_reward": np.std(total_rewards),
        "collision_rate": collision_count / n_episodes,
        "avg_lane_changes": lane_changes / n_episodes
    }

results = extended_evaluation(model, eval_env)
print("\n📊 EXTENDED EVALUATION RESULTS:")
for k, v in results.items():
    print(f"{k.upper():<20}: {v:.2f}")
```

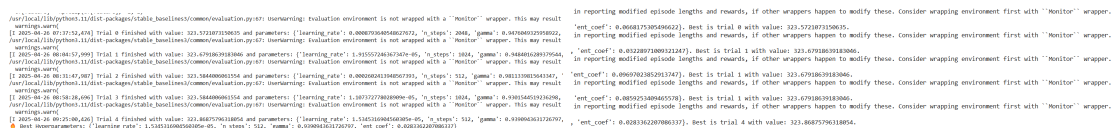
Once training is complete, an extended evaluation is performed using a custom `extended_evaluation()` function. Instead of simply measuring average rewards, this function collects deeper insights across multiple episodes: the average reward, the standard deviation of rewards, the collision rate (percentage of episodes with crashes), and the average number of lane changes. By evaluating these specific metrics, we can understand not only if the agent is fast (high rewards) but also **if it is safe (low collision rate)** and **drives smoothly (fewer unnecessary lane changes)**. This thorough evaluation ensures the trained agent is practical for real-world driving tasks, where safety and stability are just as important as speed.

e. Recording

```
%load_ext tensorboard
%tensorboard --logdir ./ppo_highway_tensorboard/

# Record video
os.system("rm -rf videos && mkdir videos")
record_env = make_env(render_mode="rgb_array")
record_env = RecordVideo(record_env, video_folder="videos", episode_trigger=lambda _: True)
obs, _ = record_env.reset()
done = False
while not done:
    action, _ = model.predict(obs, deterministic=True)
    obs, _, terminated, truncated, _ = record_env.step(action)
    done = terminated or truncated
record_env.close()
files.download("videos/rl-video-episode-0.mp4")
```

For better analysis and demonstration, the model's performance is visualized in two ways. First, Tensorboard is used to plot learning curves, rewards over time, and other training statistics. This tool gives a visual overview of how the model improved throughout the training, helping spot any issues like sudden drops in performance. Second, a video of the agent driving is recorded by wrapping the environment with `RecordVideo`, letting us watch how the vehicle behaves in action. This video serves as final proof of the model's skills and allows easy sharing of results. Recording the video is important because sometimes, even if the numerical metrics look good, the actual driving behavior might show hidden problems — so having both numbers and visuals provides full validation.

f. Result



The logs show five trials testing different combinations of hyperparameters—including learning rate, number of steps (`n_steps`), discount factor (`gamma`), and entropy coefficient (`ent_coef`)—to optimize the model's performance. The best trial (Trial 4) achieved the highest reward (323.868) with hyperparameters fine-tuned for stability and long-term reward prioritization (e.g., low learning rate of 1.53e-05 and high gamma of 0.939).

Task allocation

Name	Task
HENG ZHENG TECK	-setting up environment, choosing features) - Training DQN and SAC models - Collecting reward data for analysis - generating the output video car
Xavier Ngow Kar Yuen	Training PPO and TD3 models - Generating evaluation charts (summary table, boxplot) - Comparing results and writing findings

Reference

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal Policy Optimization Algorithms*. arXiv preprint arXiv:1707.06347.
[For PPO]

Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). *Human-level control through deep reinforcement learning*. Nature, 518(7540), 529–533.
[For DQN]

Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. Proceedings of the 35th International Conference on Machine Learning.
[For SAC]

Fujimoto, S., Hoof, H., & Meger, D. (2018). *Addressing Function Approximation Error in Actor-Critic Methods*. arXiv preprint arXiv:1802.09477.
[For TD3]

OpenAI Gym: Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., & Zaremba, W. (2016). *OpenAI Gym*. arXiv preprint arXiv:1606.01540.
[If you used OpenAI Gym environment for training/testing]

Stable-Baselines3: Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). *Stable-Baselines3: Reliable Reinforcement Learning Implementations*.
[If you used the Stable-Baselines3 library for model training]